Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Mini Project Report

On

"RoadRisk AI: A Full-Stack Intelligent Road Safety Management and Accident Severity Prediction Platform"

By

Arshita Sikaria

PRN – 1272250628

For

MCA40130 Mini Project

SY MCA – Division D

AY 2026-2027

Under The Guidance Of

Mr. Kaustubh Keer

Submitted To

Dr. Vishwanath Karad MIT World Peace University, Pune

In partial fulfillment of the requirement for the

award of Degree of

Master of Computer Applications

Submitted Through

MIT-WPU

School of Computer Science and Engineering

Department of Computer Science and Applications

Certificate

Department of Computer Science and Applications

This is to certify that, **Arshita Sikaria**, student of **SY MCA, Semester III** has completed Mini Project entitled **"RoadRisk AI: A Full-Stack Intelligent Road Safety Management and Accident Severity Prediction Platform"** under guidance of **Mr. Kaustubh Keer** in partial fulfilment of requirement for the award of **Master of Computer Applications** prescribed under

Dr. Vishwanath Karad MIT World Peace University, Pune

This project is the record of authentic work carried out by her during the academic year 2026-2027.

Mr. Kaustubh Keer
Internal Guide

Dr. P. S. Metkewar
Program Head – MCA

Dr. Anuradha S. Kanade
Program Director

Date: / /_____

Acknowledgement

I wish to express my profound gratitude to all the individuals who supported me throughout the completion of my Mini Project III. This report is a testament to their guidance, patience, and resources.

I am deeply indebted to my esteemed project guide, **Mr. Kaustubh Keer**, for his invaluable mentorship, direction, and encouragement. His advice was crucial in defining the scope and methodology of this project.

I extend my sincere thanks to the **Head of the Program, Dr. P. S. Metkewar**, and the **Program Director, Dr. Anuradha S. Kanade**, for providing the necessary facilities, academic framework, and environment conducive to advanced project work and research.

My appreciation also goes to all the **Faculty Members** of the **Department of Computer Science and Applications** for their comprehensive instruction and expertise, which equipped me with the knowledge foundational to this project.

Finally, I am grateful to my **Classmates** for their constructive feedback and discussions, which enriched my understanding and helped me refine my approach.

Name:	Arshita Sikaria
PRN:	1272250628
Place:	Pune

Date: / /_____

Declaration

I, **Arshita Sikaria**, hereby declare that this Mini Project report entitled **"RoadRisk AI: A Full-Stack Intelligent Road Safety Management and Accident Severity Prediction Platform"** is an authentic record of my own work carried out under the supervision of **Mr. Kaustubh Keer**.

I further declare that the content of this report, including all design, coding, testing, and analysis, is original and has not been submitted previously for any other degree or diploma at this or any other institution.

I take sole and full responsibility for any errors, omissions, or misrepresentations found within this report.

Signature:

Name:	Arshita Sikaria
PRN:	1272250628
Place:	Pune

Date: / /_____

Project Report Index

Index	Section Title
1	**INTRODUCTION**
1.1	EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM
1.2	SCOPE OF WORK
1.3	OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE
1.4	DESCRIPTION OF TECHNOLOGY USED
2	**PROPOSED SYSTEM**
2.1	FEASIBILITY STUDY
2.2	USER REQUIREMENTS
3	**ANALYSIS AND DESIGN**
3.1	OBJECT DIAGRAM
3.2	CLASS DIAGRAM
3.3	USE CASE DIAGRAM
3.4	SEQUENCE DIAGRAM
3.5	ACTIVITY DIAGRAM
3.6	MODULE HIERARCHY DIAGRAM
3.7	USER INTERFACE DESIGN (SCREENS ETC.)
3.8	TABLE SPECIFICATIONS
4	**INPUT AND OUTPUT SCREENS**
5	**TEST PROCEDURES AND IMPLEMENTATION**
6	**SAMPLE CODE**
7	**LIMITATIONS**
8	**FUTURE SCOPE**
9	**CONCLUSION**
10	**REFERENCES**

1. INTRODUCTION

Road traffic accidents remain one of the most pressing public safety challenges in India. The Ministry of Road Transport and Highways reports well over four lakh accidents and more than one lakh fatalities every year. A large proportion of these incidents are influenced by identifiable and recurring factors such as road geometry, weather conditions, time of day, traffic density, and vehicle type. Despite the availability of open government accident datasets, most departments still rely on manual, retrospective analysis rather than predictive, data-driven decision-making.

RoadRisk AI is a full-stack, intelligent Road Safety Management and Accident Severity Prediction Platform that combines machine learning, a web-based dashboard, and business-intelligence reporting into a single cohesive product. The system ingests historical accident records, trains and compares multiple machine learning models to predict the severity of an accident (Fatal, Serious, or Minor) from its contributing factors, and presents the resulting insights through an interactive dashboard and a Power BI executive reporting layer.

Unlike a purely academic prediction script, RoadRisk AI is architected as a deployable software product with authentication, role-based access, a relational database, REST APIs, and analytics. This reflects how such a system would be built and used in a real transport-safety or smart-city organisation.

1.1 EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM

Existing System

At present, accident data in most regions is recorded by police departments and transport authorities and published as annual reports, spreadsheets, or PDF documents. The existing approach has the following drawbacks:

- Accident data is **static and siloed** across spreadsheets and PDF reports, with no single point of access.
- Analysis is **manual and retrospective**; it explains what happened but does not predict what is likely to happen.
- There is **no automated identification of high-risk road segments** from historical patterns.
- Decision-makers have **no interactive visualisation** of seasonal, weather-based, or year-on-year trends.
- Citizens and planners have **no easy way to search** the risk profile of a specific road or locality.

Proposed System

RoadRisk AI proposes an end-to-end Road Safety Intelligence Platform. An administrator uploads and manages accident datasets; the platform automatically cleans the data, engineers features (highway corridor, area type, carriageway type, road condition, weather, time of day, day of week, vehicle type, contributing cause) and trains Random Forest and XGBoost classifiers. The better model, judged on accuracy, precision, recall and F1-score, is saved and used for live severity prediction. Registered users can submit road and environmental conditions to receive a predicted severity class together with an AI-based preventive recommendation, explore analytics dashboards, search the risk profile of a road, and download reports. A Power BI layer provides an executive-level view of the same data.

Need of the System

Traffic authorities, urban planners, and citizens currently lack a unified, accessible platform that can (a) predict the likely severity of an accident for a given set of road and environmental conditions, (b) identify high-risk road segments using historical patterns, and (c) present this information in a form that supports quick, informed decision-making. A single, intelligent, and user-friendly system is therefore needed to transform raw accident data into actionable, predictive road-safety intelligence.

Objectives of the System

1. To design and develop a full-stack web application for structured upload, storage, and management of road accident data.
2. To build and compare multiple machine learning models (Random Forest and XGBoost) for predicting accident severity, and to automatically select and persist the best-performing model.
3. To provide an interactive analytics dashboard highlighting high-risk roads, seasonal and weather-based accident trends, and year-on-year comparisons.
4. To implement secure, role-based access control distinguishing Administrators, Traffic Analysts, and Public Users.
5. To integrate a Power BI reporting layer for executive-level, exportable visual summaries of accident trends.
6. To generate downloadable, structured reports that support institutional or governmental road-safety planning.
7. To provide AI-based road safety recommendations based on predicted accident severity to support preventive decision-making.

1.2 SCOPE OF WORK

In Scope

- The system focuses on Indian national-highway accidents. The current version is trained on a published NHAI highway accident dataset covering the Pune–Solapur

(NH-9) corridor, which is the default focus of the dashboard, together with the Barwa–Adda–Panagarh (NH-2) and Nagpur / Chengapally–Walayar corridors, and it also accepts user-uploaded CSV data.

- Severity prediction is limited to three classes: Fatal, Serious, and Minor, following the standard categorisation used in Indian road-accident reporting.
- The platform covers data upload, model training and comparison, prediction, dashboard analytics, Power BI reporting, report generation, and administrative management.
- AI-based road safety recommendations are provided according to the predicted accident severity to assist authorities in preventive planning.
- An India Statistics module presents real state-, UT- and city-wise road accident figures (cases, injured, died) from the NCRB report "Accidental Deaths & Suicides in India" (Table 1A.2).

Out of Scope

- Real-time GPS/IoT sensor integration.
- Live traffic-camera analysis and video-based accident detection.
- Native mobile application development (the web interface is responsive instead).

These items are not part of this mini project but are noted as future enhancements in Section 8.

1.3 OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE

Table 1.1 – Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB	16 GB (for faster model training)
Storage	256 GB HDD	512 GB SSD
Display	1366 × 768 resolution	1920 × 1080 resolution
Network	Broadband internet connection	Broadband internet connection

Table 1.2 – Software Requirements

Category	Software
Operating System	Windows 10 / 11 (also runs on Linux and macOS)
Programming Language	Python 3.10 or higher
Web Framework	Django and Django REST Framework
ML Libraries	Scikit-learn, XGBoost, Pandas, NumPy, Joblib
Database	MySQL / PostgreSQL
Frontend	HTML5, CSS3, JavaScript, Bootstrap, Chart.js
Business Intelligence	Microsoft Power BI Desktop

Category	Software
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest versions)
Development Tools	VS Code, Jupyter Notebook, Postman, Git and GitHub

1.4 DESCRIPTION OF TECHNOLOGY USED

Python: A high-level, general-purpose language used for both the backend and the machine learning pipeline. Its large ecosystem of data science libraries allows data processing, model training, and web development to be done in one language.

Django and Django REST Framework: Django is a high-level web framework that provides an ORM, authentication, an admin interface, and a clean project structure. Django REST Framework (DRF) extends it with serializers, viewsets, and permission classes used to expose the prediction, analytics, and reporting features as secure REST APIs.

Scikit-learn: A machine learning library providing preprocessing utilities, model selection tools, evaluation metrics, and the Random Forest implementation used in this project.

Random Forest: An ensemble learning method that builds many decision trees on random subsets of the data and features and combines their votes. It is robust to noise and overfitting and handles categorical-heavy tabular data well (Breiman, 2001).

XGBoost: An optimised gradient-boosted decision tree library that builds trees sequentially, each correcting the errors of the previous ones. It is widely used for structured data and is compared against Random Forest in this project (Chen and Guestrin, 2016).

Pandas and NumPy: Libraries used for loading CSV data, cleaning missing values, encoding categorical variables, and numerical computation.

MySQL / PostgreSQL: Relational database systems used to store users, accident records, predictions, reports, and recommendations. The same database is connected to Power BI for reporting.

HTML5, CSS3, JavaScript, Bootstrap and Chart.js: The frontend stack. Bootstrap provides a responsive layout, and Chart.js renders interactive charts such as monthly trends and weather-wise distribution.

Microsoft Power BI: A business-intelligence tool used to build the executive dashboard, connecting directly to the platform's database to visualise accident trends for stakeholders.

Git, GitHub, VS Code, Jupyter Notebook and Postman: Version control, source hosting, code editing, model experimentation, and API testing tools used during development.

2. PROPOSED SYSTEM

RoadRisk AI is proposed as a layered web platform in which the presentation, application, machine-learning, data, and analytics concerns are separated so that each layer can be developed, tested, and scaled independently. Figure 2.1 shows the overall architecture, and Figure 2.2 shows the end-to-end workflow from user authentication to analytics and reporting.

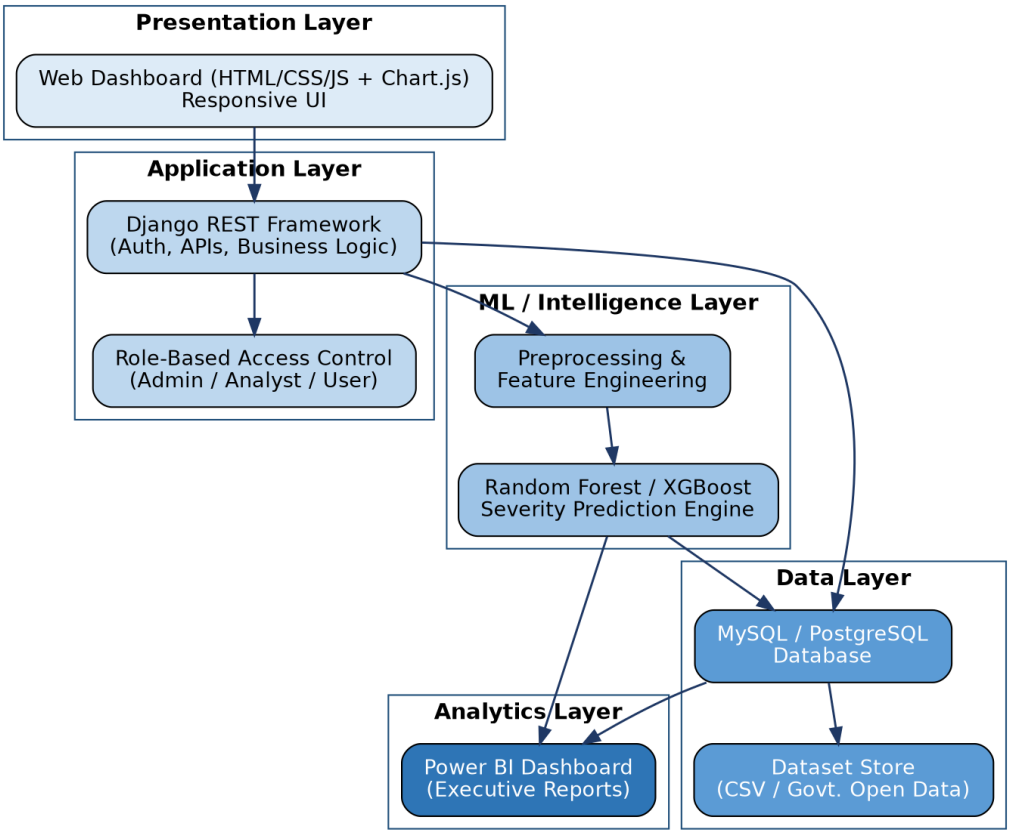


Fig 2.1 – System Architecture of RoadRisk AI

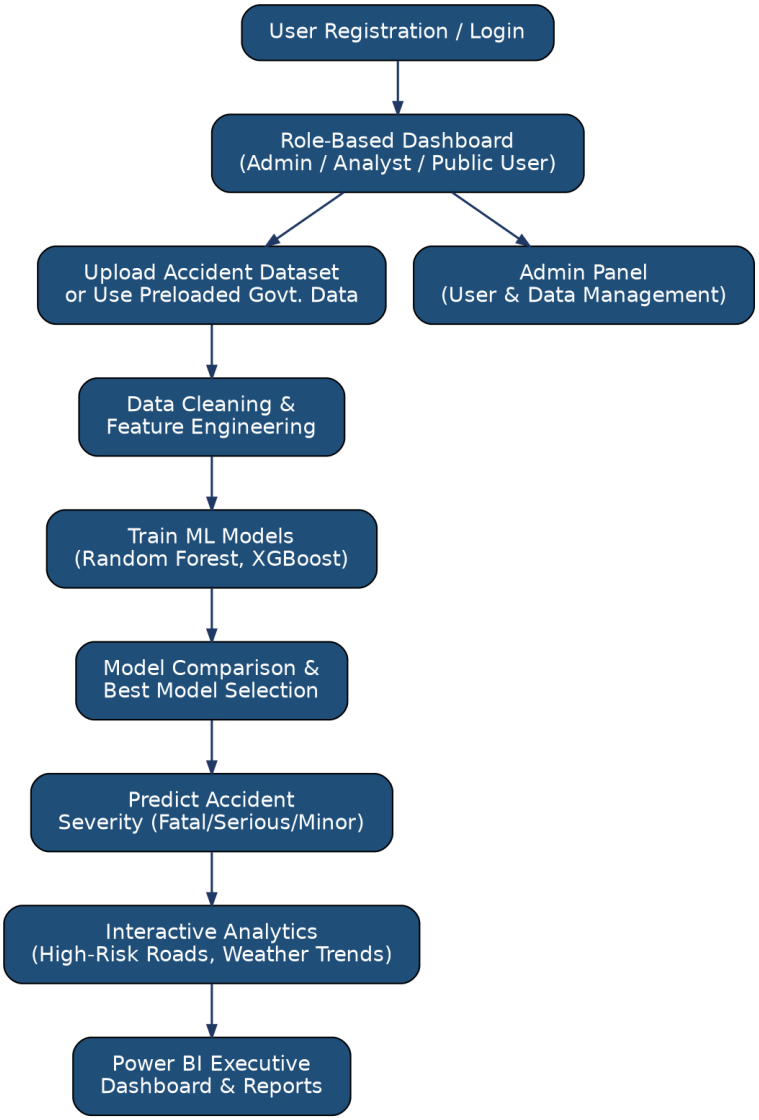


Fig 2.2 – System Work-Flow Diagram of RoadRisk AI

2.1 FEASIBILITY STUDY

Technical Feasibility. The project uses mature, open-source technologies (Python, Django, Scikit-learn, XGBoost, MySQL/PostgreSQL) that are well documented and run on an ordinary laptop. Open accident datasets are available from government portals. The required skills in Python, web development, and machine learning are available, so the project is technically feasible.

Economic Feasibility. All development tools and libraries are free and open source, and Power BI Desktop is free for individual use. No special hardware is required, so the development cost is minimal. For an organisation, the benefit of predictive road-safety insight outweighs the small deployment and hosting cost.

Operational Feasibility. The web interface is simple, responsive, and role-based, so administrators, analysts, and the public can use it with minimal training. Reports are downloadable in familiar PDF and Excel formats.

Schedule Feasibility. The project is divided into clearly defined modules (authentication, data management, ML engine, dashboard, reporting, Power BI, admin, recommendation) that can be built incrementally and completed within the mini project timeline.

Data Feasibility. A published research dataset of NHAI highway accidents (Zenodo, CC BY 4.0) and the NCRB state-wise accident statistics are openly available, and the system also accepts user-uploaded CSV files, so sufficient data is available for training and evaluation.

2.2 USER REQUIREMENTS

Functional Requirements

Table 2.1 – Functional Requirements by Actor

Actor	Requirements
Guest User	Search the risk profile of a road or locality. Register and log in to the system.
Registered User (Analyst / Public)	Submit road and environmental conditions and view the predicted severity. View AI-based safety recommendations. View the analytics dashboard (trends, weather distribution, high-risk roads). Generate and download reports.
Administrator	Upload, validate, approve, and manage accident datasets. Train and compare ML models and select the best one. Manage users and their roles. View the Power BI executive dashboard.

Non-Functional Requirements

- **Security:** passwords are stored as hashes; role-based access control protects sensitive features; APIs require authentication tokens.
- **Performance:** a prediction request should return its result within a few seconds because the trained model is loaded from disk rather than retrained.
- **Usability:** a clean, responsive interface that works on desktops, tablets, and phones.
- **Reliability:** input validation for forms and uploaded files to prevent invalid data entering the database.
- **Maintainability:** modular code organised as separate Django apps, with version control through Git.
- **Scalability:** the layered architecture allows the ML engine and the database to be scaled independently.

3. ANALYSIS AND DESIGN

This section presents the analysis and design models of RoadRisk AI using UML diagrams, the module structure, the user interface design, and the database table specifications.

3.1 OBJECT DIAGRAM

The object diagram below shows a snapshot of the system at run time: an approved Dataset (d1) contains Accident records such as a1; a Traffic Analyst (u1) submits road conditions and receives a Prediction (p1) from the best trained model (m1); the prediction is summarised in a Report (r1) and produces a Recommendation (rec1).

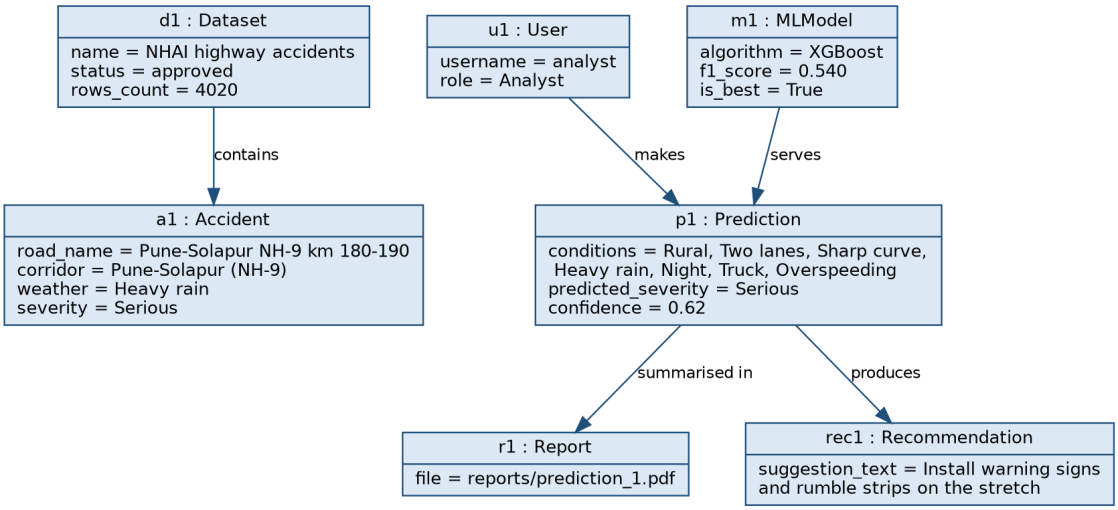


Fig 3.1 – Object Diagram of RoadRisk AI

3.2 CLASS DIAGRAM

The class diagram shows the main classes of the system with their attributes, operations, and relationships. A User makes many Predictions; each Prediction is served by a trained MLModel and is linked to one Report and one or more Recommendations (the input conditions are stored with the prediction, and a prediction can optionally refer to an Accident record). A Dataset uploaded by an Administrator contains many Accident records.

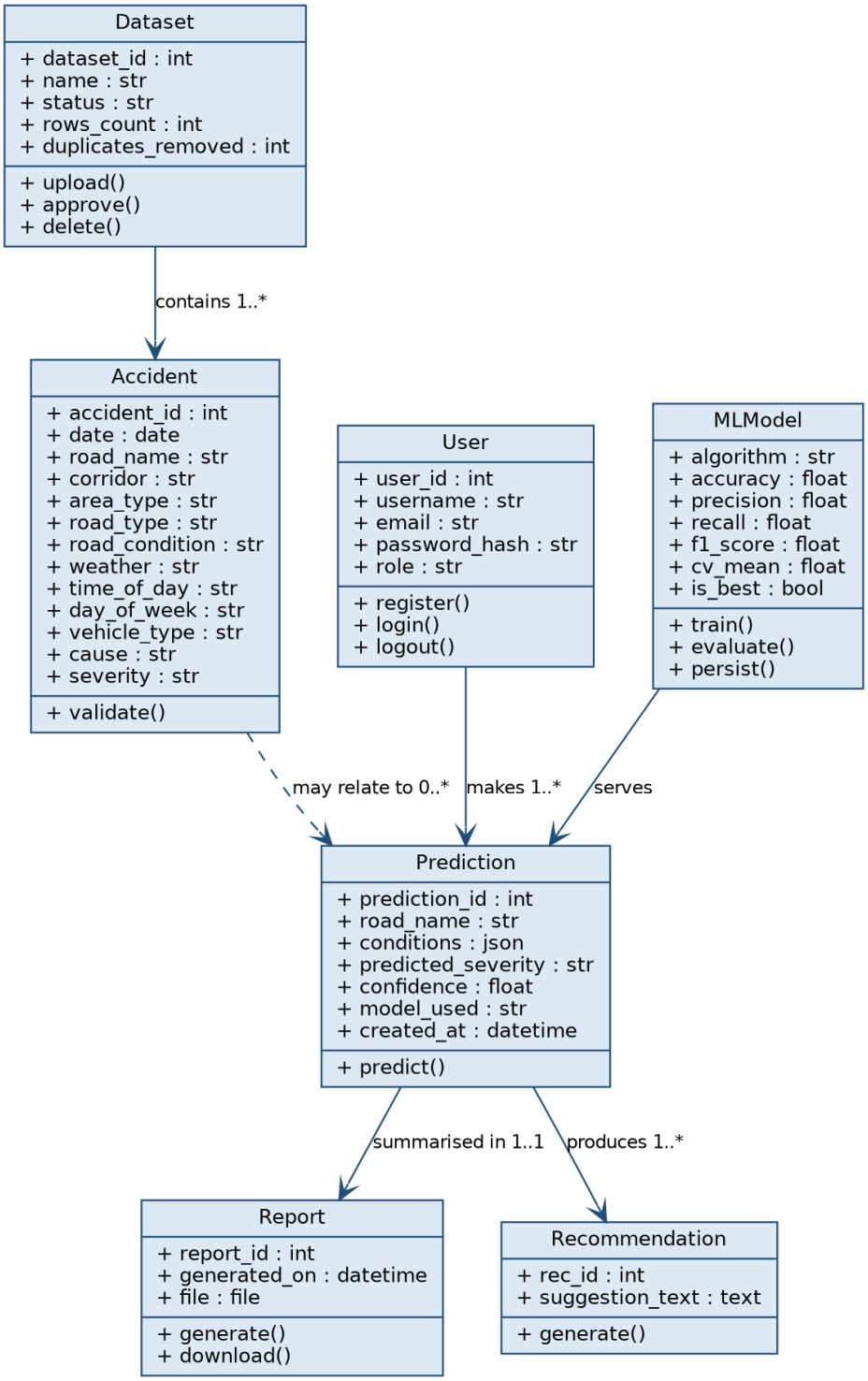


Fig 3.2 – Class Diagram of RoadRisk AI

3.3 USE CASE DIAGRAM

The use case diagram illustrates the three primary actors of the system and the functionality available to each. The Guest User can search road risk and register or log in. The Registered User can additionally predict accident severity, view AI safety recommendations, view the analytics dashboard, and generate or download reports. The Administrator can upload and manage datasets, train and compare ML models, manage users, and view Power BI insights.

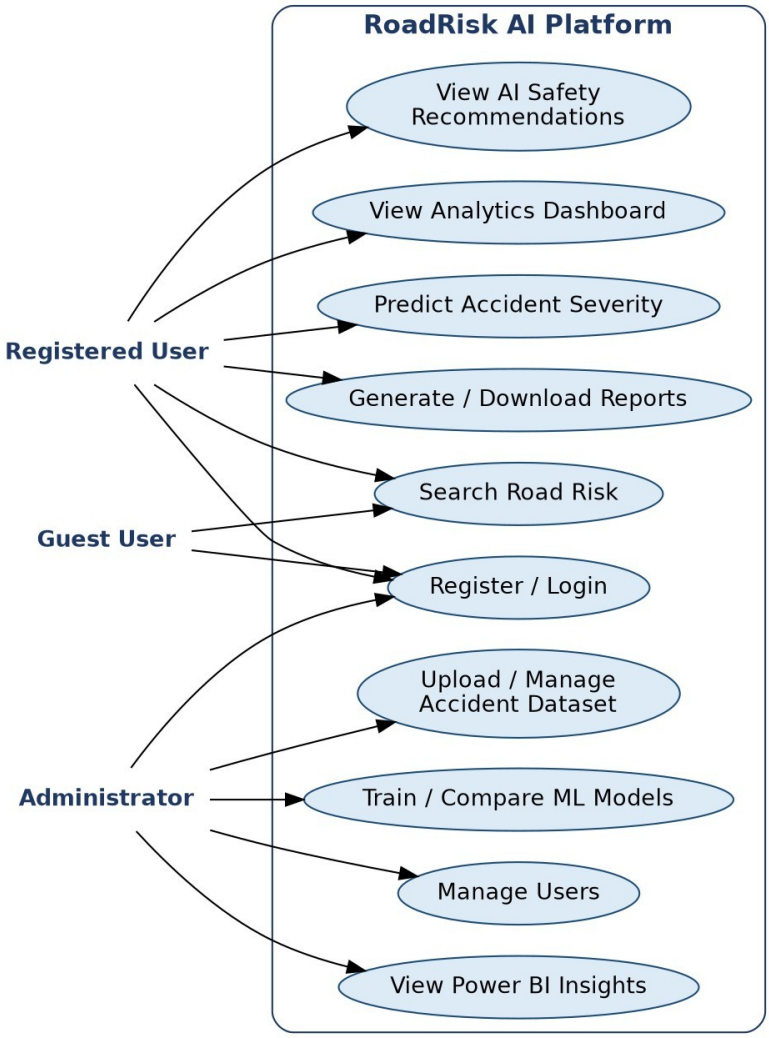


Fig 3.3 – Use-Case Diagram of RoadRisk AI

3.4 SEQUENCE DIAGRAM

The sequence diagram shows the interaction between the user, web dashboard, Django REST API, ML engine, and database while predicting accident severity. The request is authenticated and validated by the API, the features are preprocessed and passed to the saved best model, and the predicted severity and recommendation are stored and returned to the user.

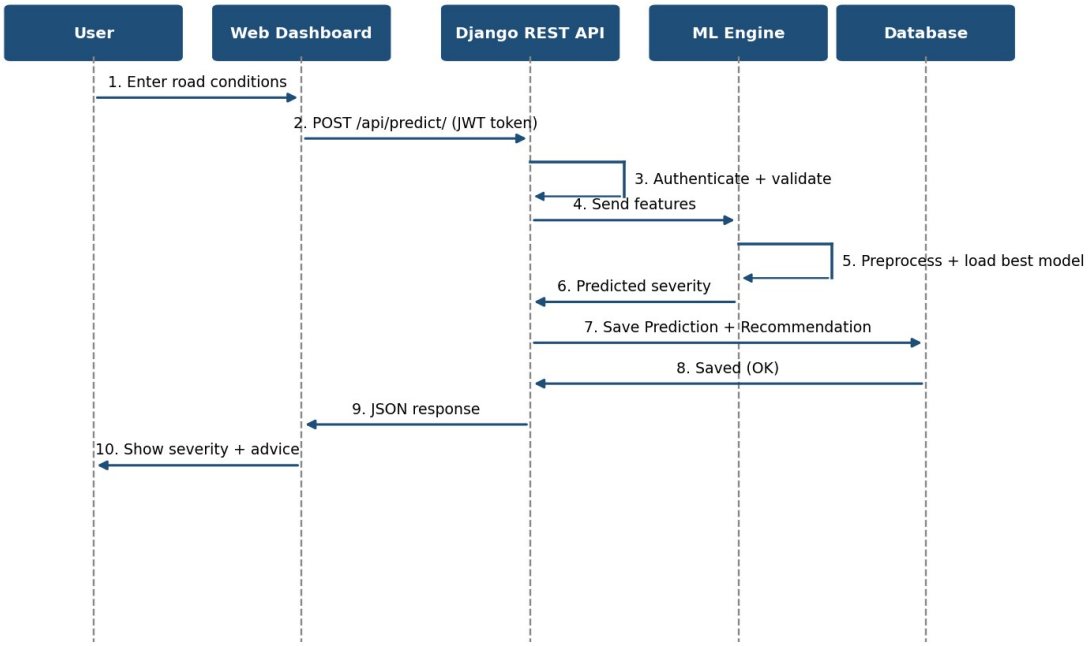


Fig 3.4 – Sequence Diagram: Predict Accident Severity

3.5 ACTIVITY DIAGRAM

The activity diagram models the flow of activities for a severity prediction, from login to the optional generation of a report, including the decision for invalid input.

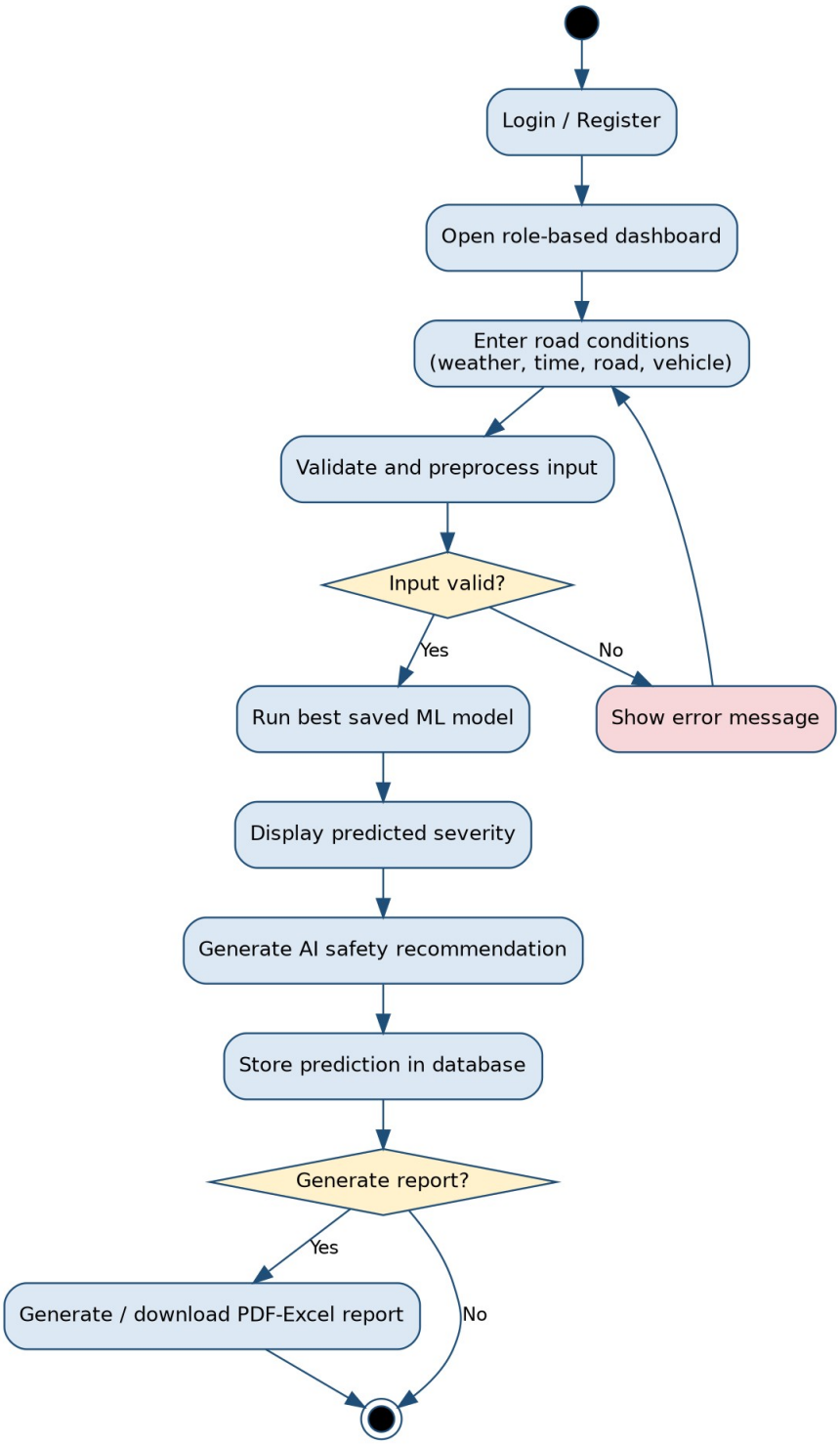


Fig 3.5 – Activity Diagram: Severity Prediction

3.6 MODULE HIERARCHY DIAGRAM

The platform is divided into nine modules, each with its own sub-functions, as shown in Figures 3.6(a) and 3.6(b).

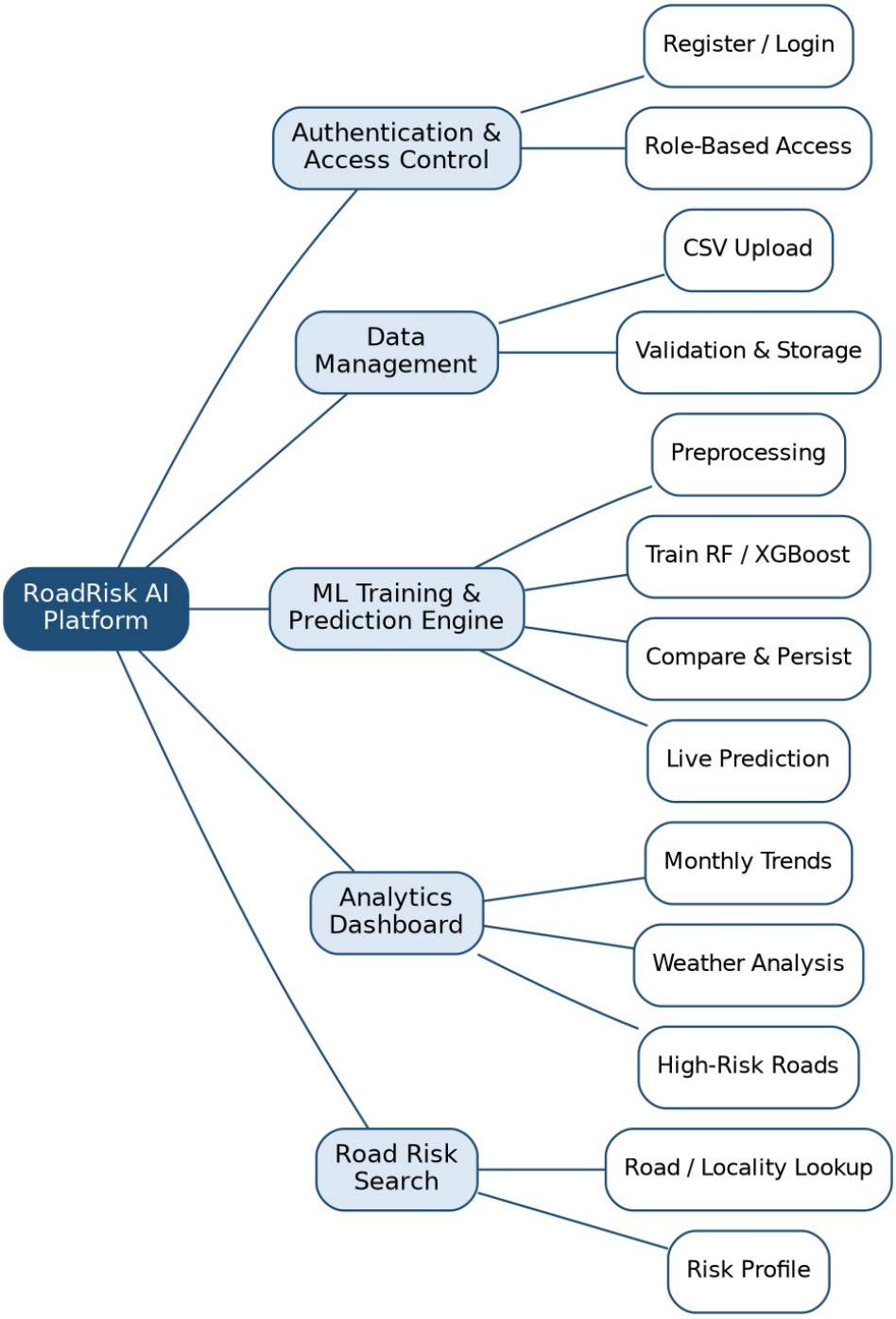


Fig 3.6(a) – Module Hierarchy Diagram of RoadRisk AI (Part 1)

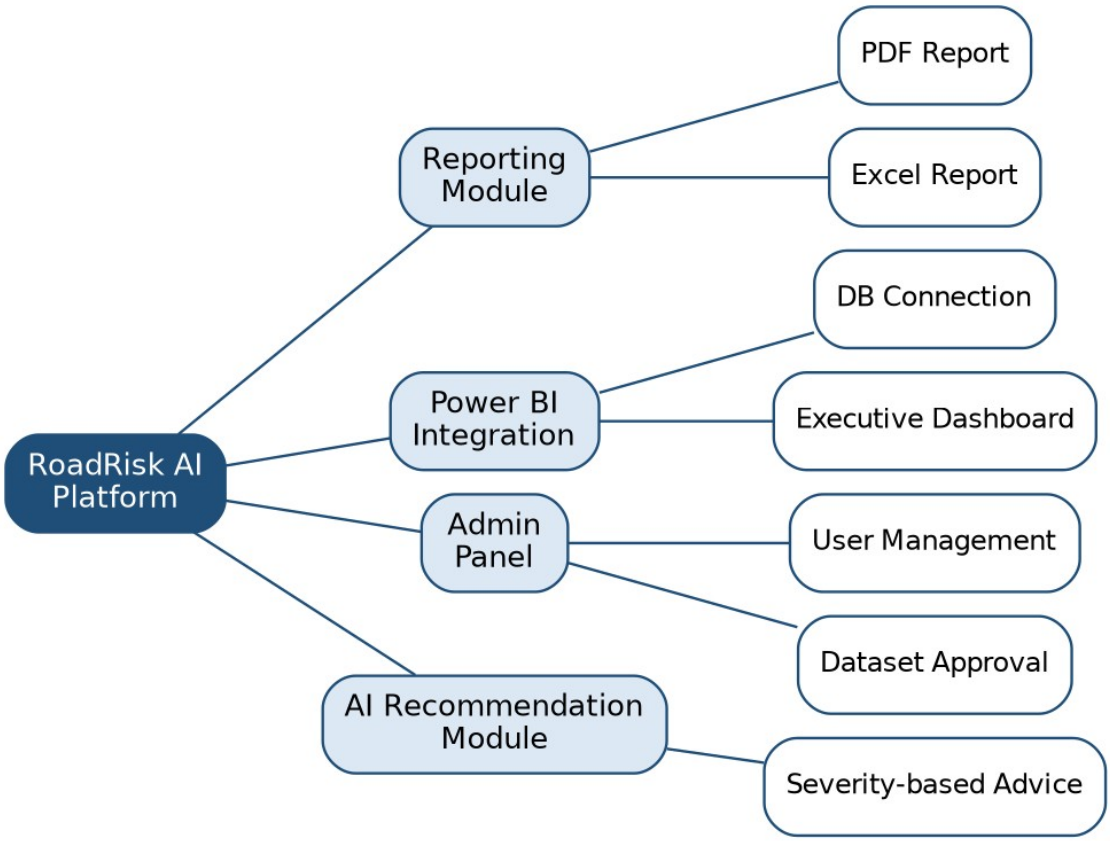


Fig 3.6(b) – Module Hierarchy Diagram of RoadRisk AI (Part 2)

Table 3.1 – Module Descriptions

Module	Description
Authentication & Access Control	Secure registration and login with role-based access for Admin, Analyst, and Public User roles.
Data Management	Upload, validate, and store accident datasets; supports user-uploaded CSVs and a preloaded reference dataset.
ML Training & Prediction Engine	Preprocesses data, trains Random Forest and XGBoost models, compares performance, and serves live severity predictions.
Analytics Dashboard	Interactive charts for monthly trends, weather-wise distribution, and high-risk road rankings.
Road Risk Search	Lets users search a road or locality and view its historical risk profile.
Reporting Module	Generates downloadable PDF/Excel reports summarising predictions and analytics.
Power BI Integration	Executive dashboard connected to the same database for stakeholder-facing reporting.
Admin Panel	Dataset approval, user management, and system monitoring.
AI Recommendation Module	Provides preventive road safety recommendations based on predicted severity and road conditions.

3.7 USER INTERFACE DESIGN (SCREENS ETC.)

The interface follows a modern dark dashboard design: a fixed left sidebar for navigation (the items change with the user's role), a top bar with the page title and filters, and rounded cards for data. Severity is colour-coded consistently across the system (green = Minor, amber = Serious, red = Fatal). Bootstrap 5 provides the responsive grid, Chart.js draws the charts, and all libraries are bundled with the project so that the system also works without an internet connection. On small screens the sidebar collapses behind a menu button.

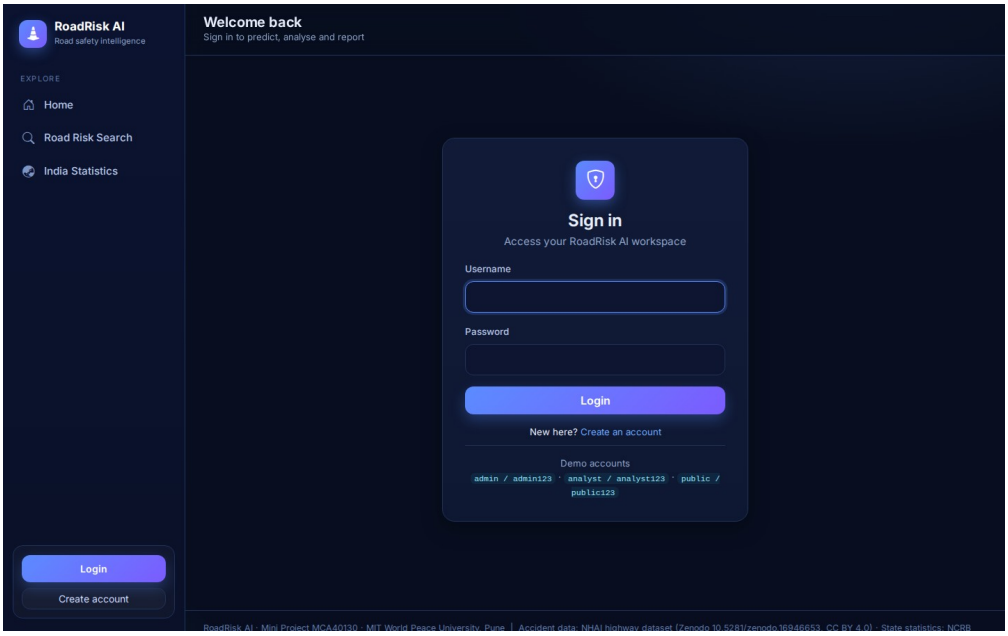


Fig 3.7 – Login screen

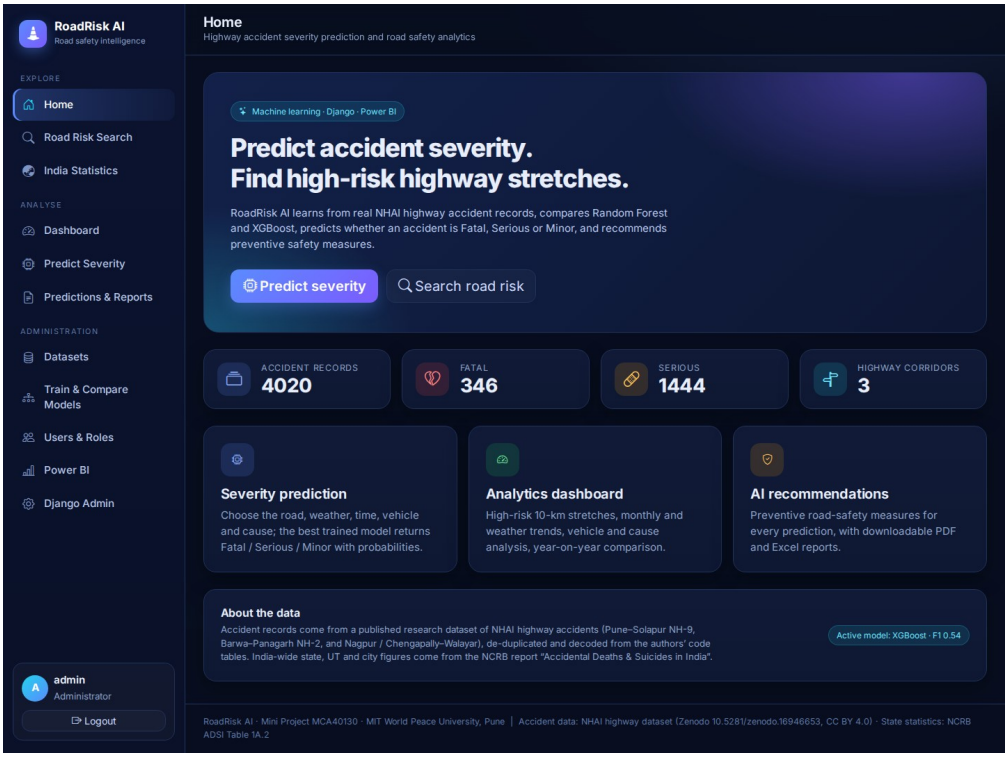


Fig 3.8 – Home screen

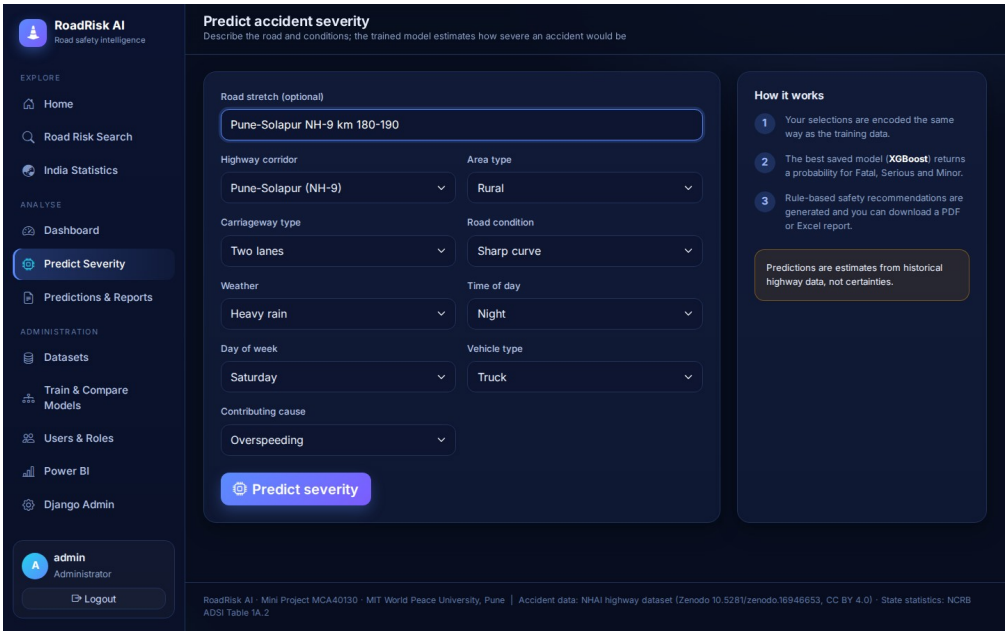


Fig 3.9 – Severity prediction screen



Fig 3.10 – Responsive (mobile) dashboard

3.8 TABLE SPECIFICATIONS

The Entity-Relationship diagram below outlines the database design. A User makes many Predictions (1:M); a Dataset contains many Accident records (1:M); a Prediction has one Report (1:1) and many Recommendations (1:M). A Prediction can optionally refer to an Accident record. State, UT and city statistics from NCRB are stored in a separate RegionStat table.

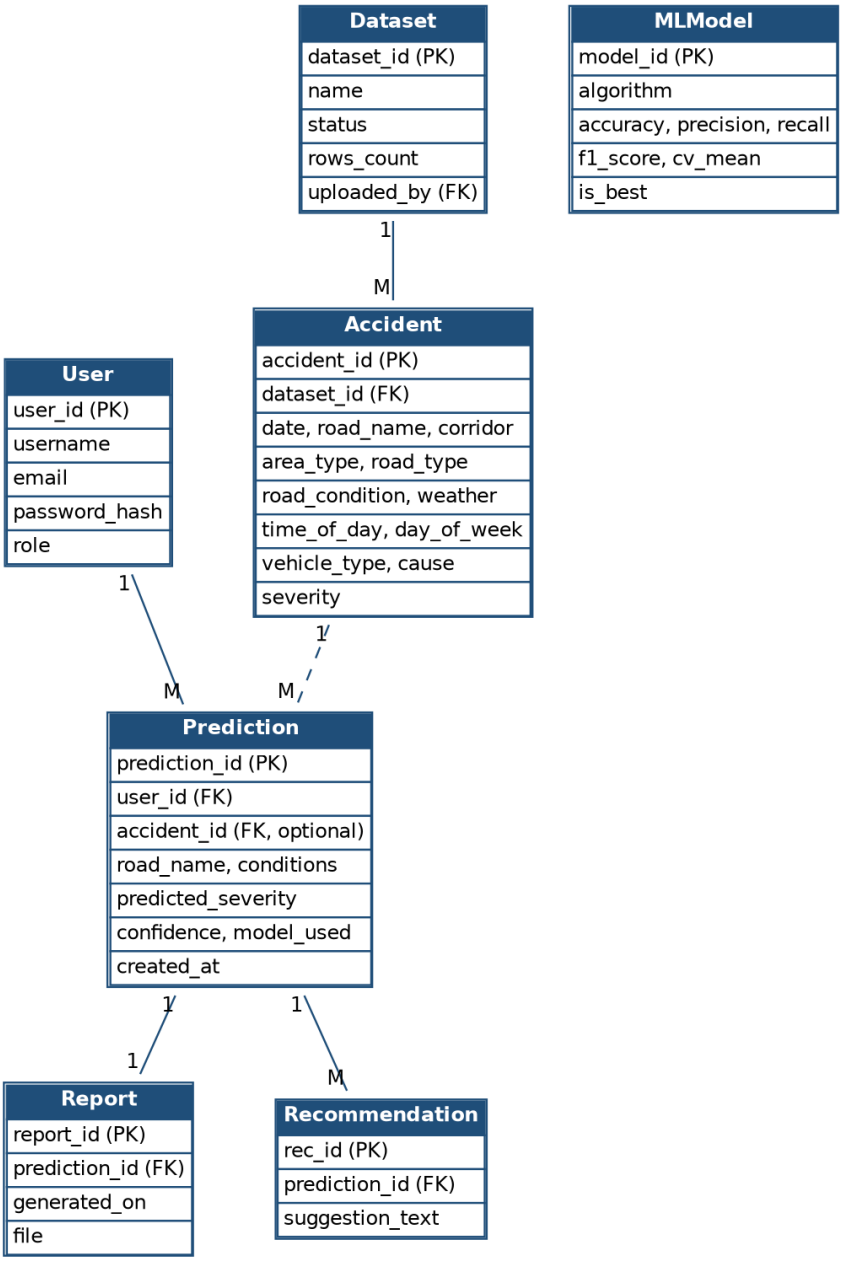


Fig 3.11 – Entity-Relationship (ER) Diagram of RoadRisk AI

Table 3.2 – User

Field	Data Type	Constraint	Description
user_id	INT	Primary Key, Auto-increment	Unique user identifier

Field	Data Type	Constraint	Description
username	VARCHAR(150)	UNIQUE, NOT NULL	Login name
email	VARCHAR(254)	NOT NULL	Email address
password_hash	VARCHAR(128)	NOT NULL	Hashed password (Django PBKDF2)
role	VARCHAR(10)	NOT NULL	admin / analyst / public

Table 3.3 – Dataset

Field	Data Type	Constraint	Description
dataset_id	INT	Primary Key, Auto-increment	Unique dataset identifier
name	VARCHAR(150)	NOT NULL	Dataset name
file	VARCHAR(100)		Stored CSV file
uploaded_by	INT	Foreign Key → User	Administrator who uploaded it
status	VARCHAR(10)	NOT NULL	pending / approved
rows_count	INT	NOT NULL	Records stored
duplicates_removed	INT	NOT NULL	Exact duplicate rows dropped

Table 3.4 – Accident

Field	Data Type	Constraint	Description
accident_id	INT	Primary Key, Auto-increment	Unique accident record identifier
dataset_id	INT	Foreign Key → Dataset	Source dataset
date	DATE	NULL allowed	Date of accident
road_name	VARCHAR(150)	NOT NULL, indexed	Road or 10-km stretch name
corridor	VARCHAR(80)	NOT NULL	Highway corridor
area_type	VARCHAR(30)	NOT NULL	Urban / Rural
road_type	VARCHAR(40)	NOT NULL	Carriageway type (lanes)
road_condition	VARCHAR(40)	NOT NULL	Straight, curve, incline, etc.

Field	Data Type	Constraint	Description
weather	VARCHAR(30)	NOT NULL	Weather at the time of accident
time_of_day	VARCHAR(20)	NOT NULL	Morning / Afternoon / Evening / Night
day_of_week	VARCHAR(12)	NOT NULL	Weekday name
vehicle_type	VARCHAR(40)	NOT NULL	Vehicle involved
cause	VARCHAR(60)	NOT NULL	Contributing cause
severity	VARCHAR(10)	NOT NULL	Fatal / Serious / Minor

Table 3.5 – MLModel

Field	Data Type	Constraint	Description
model_id	INT	Primary Key, Auto-increment	Unique model run identifier
algorithm	VARCHAR(40)	NOT NULL	Random Forest or XGBoost
accuracy, precision, recall	FLOAT	NOT NULL	Hold-out test metrics
f1_score, f1_macro	FLOAT	NOT NULL	Weighted and macro F1
cv_mean, cv_std	FLOAT	NOT NULL	5-fold cross-validation accuracy
baseline_accuracy	FLOAT	NOT NULL	Majority-class baseline
details	JSON		Confusion matrix, per-class results, feature importance
is_best	BOOLEAN	NOT NULL	Model used for live prediction
trained_by	INT	Foreign Key → User	Administrator who trained it

Table 3.6 – Prediction

Field	Data Type	Constraint	Description
prediction_id	INT	Primary Key, Auto-increment	Unique prediction identifier
user_id	INT	Foreign Key → User	User who made the prediction
accident_id	INT	Foreign Key → Accident, NULL allowed	Optional related accident record
road_name	VARCHAR(150)		Optional road stretch

Field	Data Type	Constraint	Description
conditions	JSON	NOT NULL	Input conditions used for the prediction
predicted_severity	VARCHAR(10)	NOT NULL	Fatal / Serious / Minor
confidence, probabilities	FLOAT / JSON	NOT NULL	Class probabilities
model_used	VARCHAR(40)	NOT NULL	Model that produced it
created_at	DATETIME	NOT NULL	Time of prediction

Table 3.7 – Report

Field	Data Type	Constraint	Description
report_id	INT	Primary Key, Auto-increment	Unique report identifier
prediction_id	INT	Foreign Key → Prediction, UNIQUE	Prediction summarised
generated_on	DATETIME	NOT NULL	Report generation time
file	VARCHAR(100)	NOT NULL	Stored PDF file

Table 3.8 – Recommendation

Field	Data Type	Constraint	Description
rec_id	INT	Primary Key, Auto-increment	Unique recommendation identifier
prediction_id	INT	Foreign Key → Prediction	Related prediction
suggestion_text	TEXT	NOT NULL	Preventive safety recommendation

Table 3.9 – RegionStat (NCRB statistics)

Field	Data Type	Constraint	Description
id	INT	Primary Key, Auto-increment	Record identifier
category, name	VARCHAR	UNIQUE together	State / UT / City and its name
road_cases, road_injured, road_died	INT	NOT NULL	Road accident cases, persons injured, persons died
total_cases, total_died	INT	NOT NULL	All traffic accidents

4. INPUT AND OUTPUT SCREENS

This section presents the input and output screens of the implemented system, captured from the running application (Pune–Solapur corridor selected on the dashboard). For each screen the inputs given by the user and the output produced are described.

4.1 Login and Registration Screens (Input)

Input: username and password (and email for registration). Output: redirect to the dashboard, or an error message for wrong credentials. New users are created as Public Users.

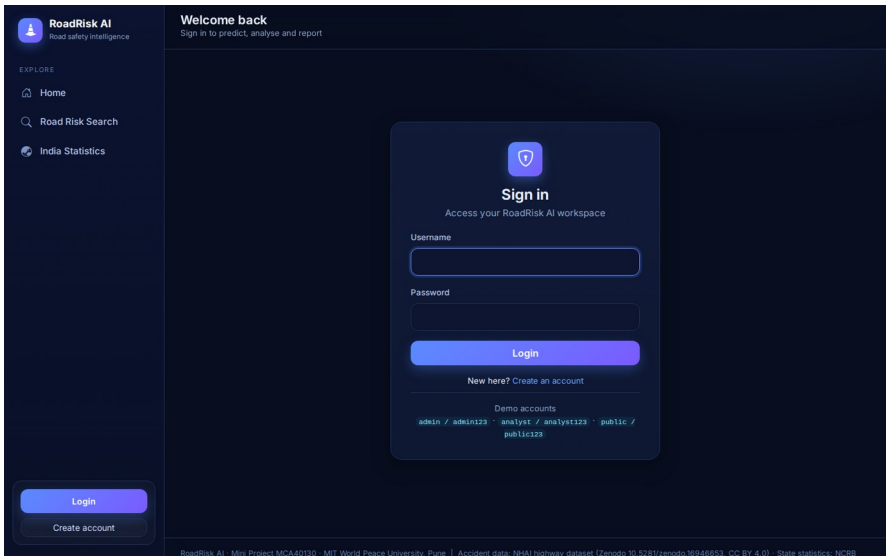


Fig 4.1 – Login screen

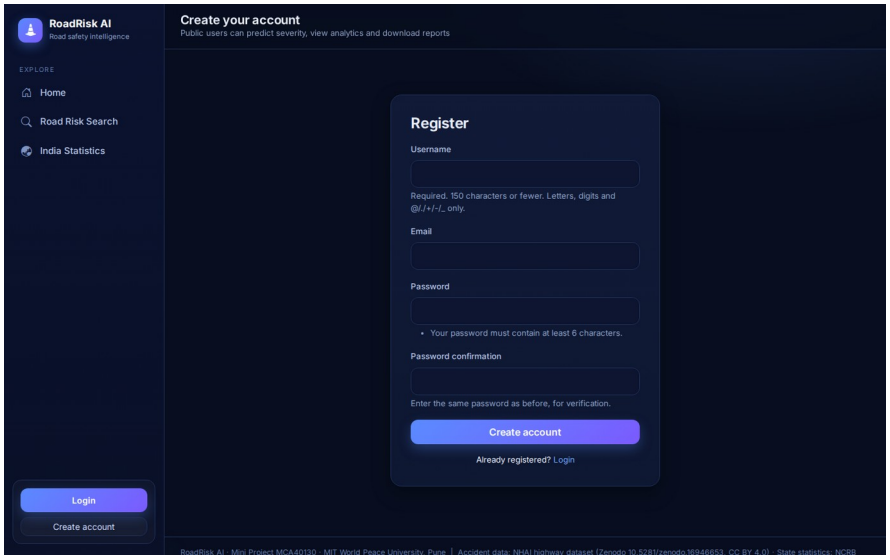


Fig 4.2 – Registration screen

4.2 Dataset Upload Screen (Input)

Input: dataset name and a CSV file. Output: validation result, number of records stored, duplicates removed, and the dataset list with approve / delete actions (Administrator only).

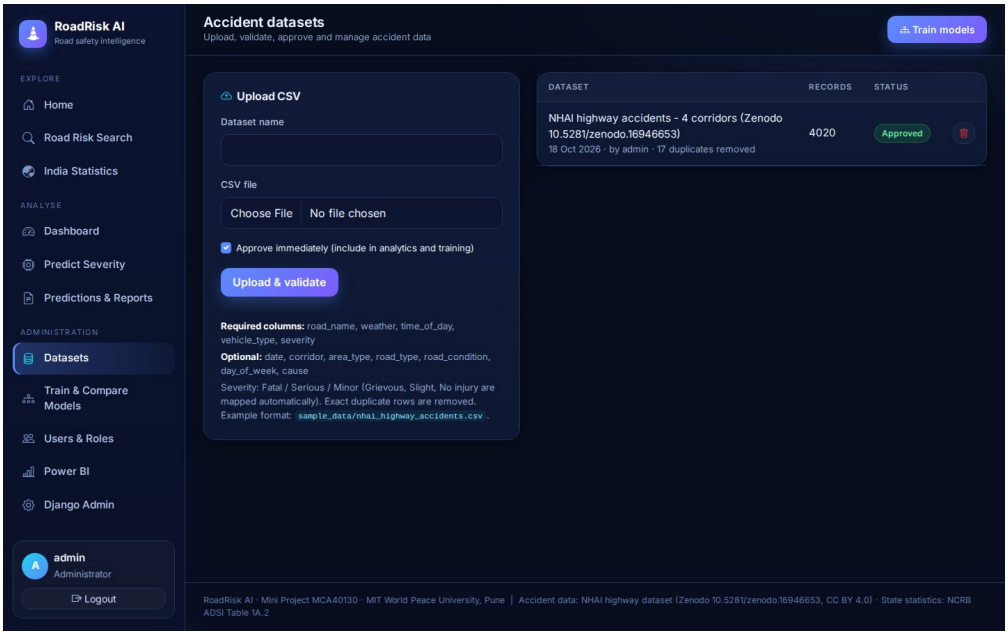


Fig 4.3 – Dataset upload and management

4.3 Model Training and Comparison Screen (Output)

Output: accuracy, precision, recall, weighted and macro F1-score and cross-validation accuracy of Random Forest and XGBoost, the majority-class baseline, the confusion matrix, per-class results and feature importance. The best model is marked active.

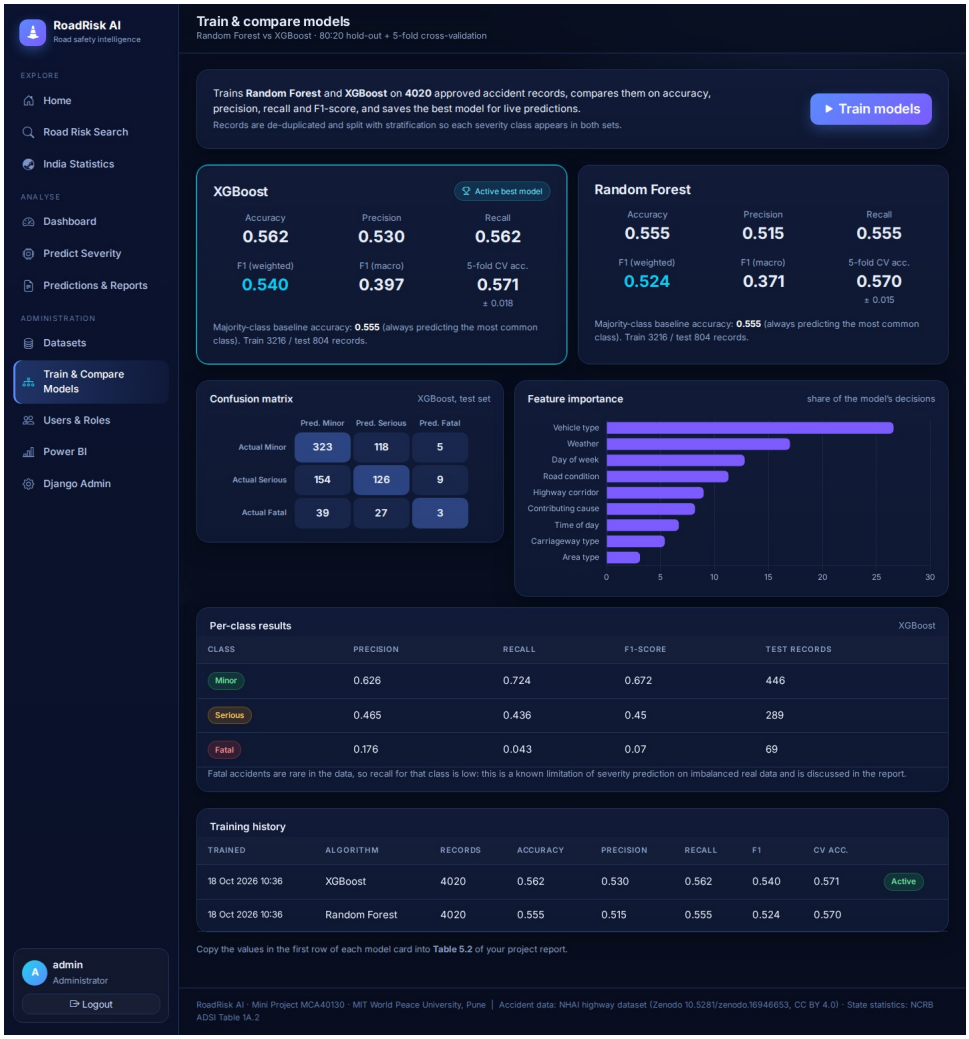


Fig 4.4 – Train and compare models

4.4 Severity Prediction Screen (Input)

Input: road stretch (optional), highway corridor, area type, carriageway type, road condition, weather, time of day, day of week, vehicle type and contributing cause.

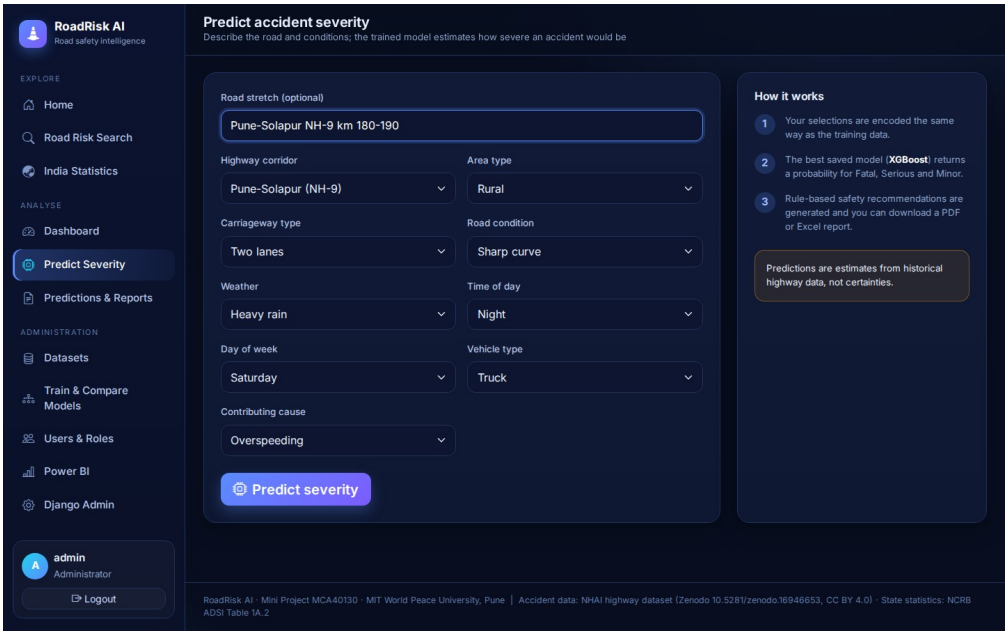


Fig 4.5 – Severity prediction form

4.5 Prediction Result Screen (Output)

Output: predicted severity class, confidence, probability of each class, the input conditions, AI road safety recommendations and buttons to download the PDF and Excel reports.



Fig 4.6 – Prediction result with recommendations

4.6 Analytics Dashboard (Output)

Output: key figures, monthly trend, severity split, year-on-year comparison, weather, time of day, day of week, vehicle type, contributing causes and the ranked high-risk 10-km stretches, filterable by corridor and year.



Fig 4.7 – Analytics dashboard (top)



Fig 4.8 – Analytics dashboard (bottom)

4.7 Road Risk Search Screen (Input / Output)

Input: road, stretch or corridor name. Output: matching stretches with accident counts and risk level, and a risk profile of the selected stretch.



Fig 4.9 – Road risk search results

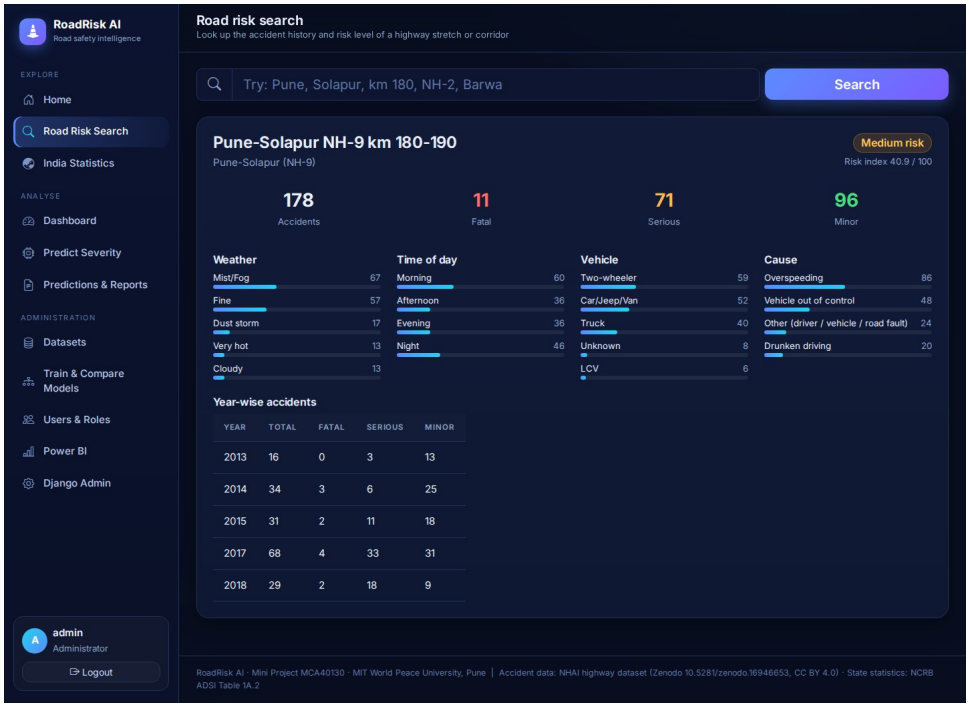


Fig 4.10 – Road risk profile

4.8 Report Generation Screen (Output)

Output: list of predictions with PDF and Excel download links, and the downloadable PDF report with inputs, predicted severity and recommendations.

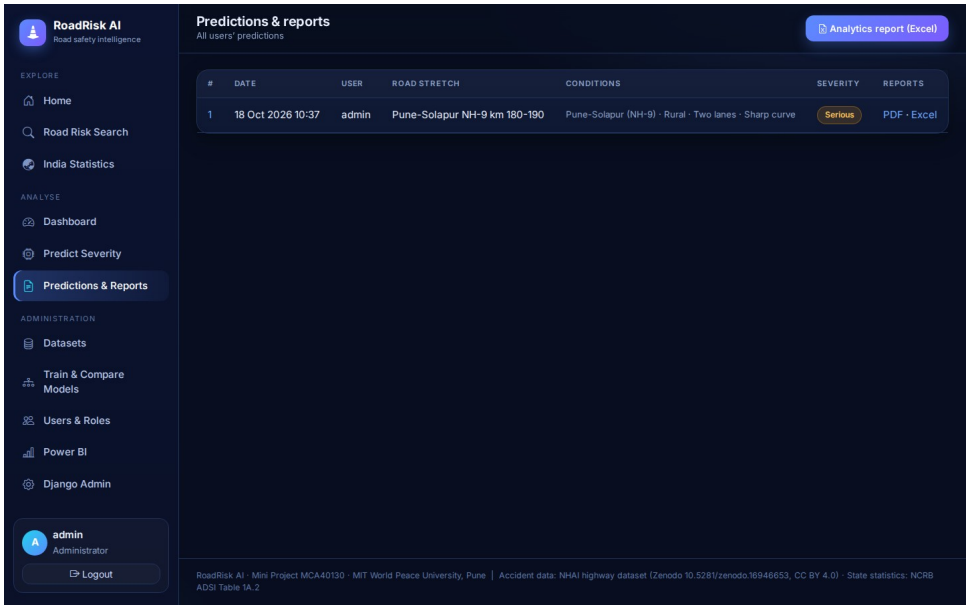
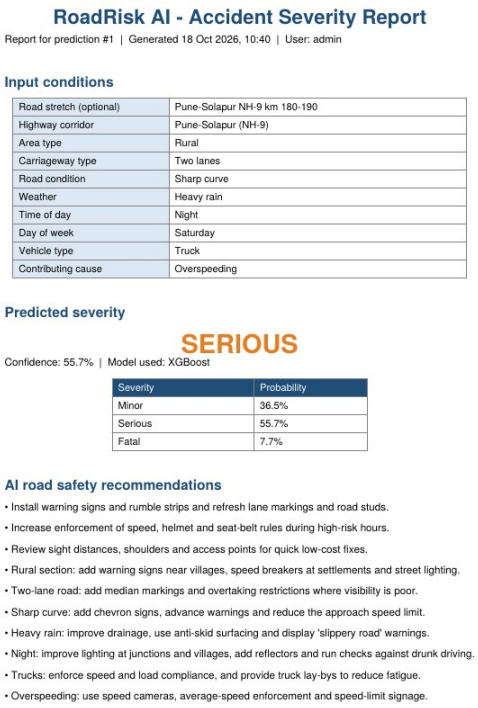


Fig 4.11 – Predictions and reports



RoadRisk AI - Accident Severity Report

Report for prediction #1 | Generated 18 Oct 2026, 10:40 | User: admin

Input conditions

Road stretch (optional)	Pune-Solapur NH-9 km 180-190
Highway corridor	Pune-Solapur (NH-9)
Area type	Rural
Carriageway type	Two lanes
Road condition	Sharp curve
Weather	Heavy rain
Time of day	Night
Day of week	Saturday
Vehicle type	Truck
Contributing cause	Overspeeding

Predicted severity

SERIOUS

Confidence: 55.7% | Model used: XGBoost

Severity	Probability
Minor	36.5%
Serious	55.7%
Fatal	7.7%

AI road safety recommendations

- Install warning signs and rumble strips and refresh lane markings and road studs.
- Increase enforcement of speed, helmet and seat-belt rules during high-risk hours.
- Review sight distances, shoulders and access points for quick low-cost fixes.
- Rural section: add warning signs near villages, speed breakers at settlements and street lighting.
- Two-lane road: add median markings and overtaking restrictions where visibility is poor.
- Sharp curve: add chevron signs, advance warnings and reduce the approach speed limit.
- Heavy rain: improve drainage, use anti-skid surfacing and display 'slippery road' warnings.
- Night: improve lighting at junctions and villages, add reflectors and run checks against drunk driving.
- Trucks: enforce speed and load compliance, and provide truck lay-bys to reduce fatigue.
- Overspeeding: use speed cameras, average-speed enforcement and speed-limit signage.

Fig 4.12 – Sample PDF report

4.9 Power BI Executive Dashboard (Output)

Output: the Power BI page of the application, which provides the export CSV, the build steps and the embedded report, followed by the executive dashboard built in Power BI Desktop.

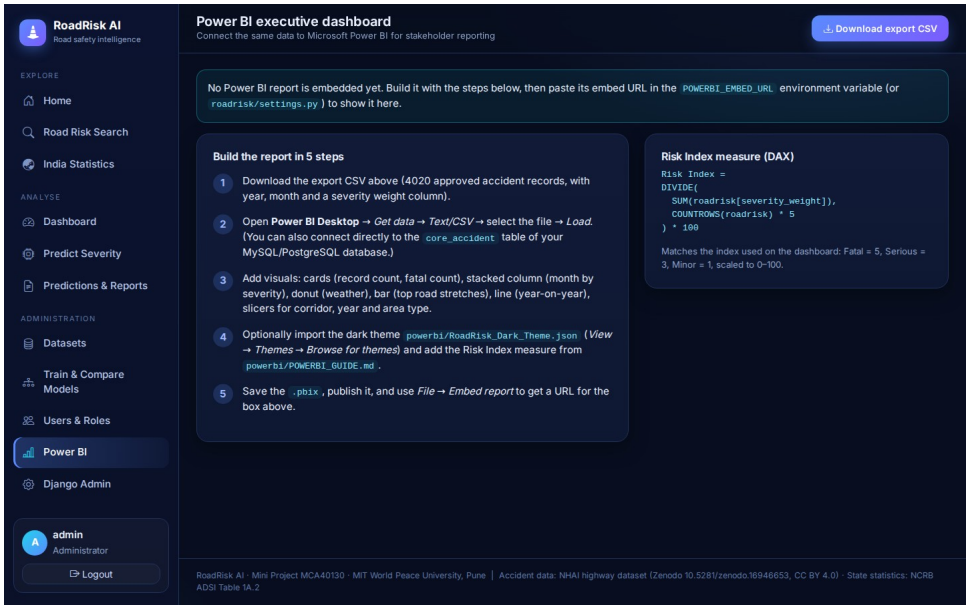


Fig 4.13 – Power BI page of the application

[Insert screenshot: Power BI report built in Power BI Desktop]

Fig 4.14 – Executive dashboard in Power BI Desktop

4.10 Admin Panel (Input / Output)

Input: role changes for users. Output: updated user list with roles (Administrator, Traffic Analyst, Public User).

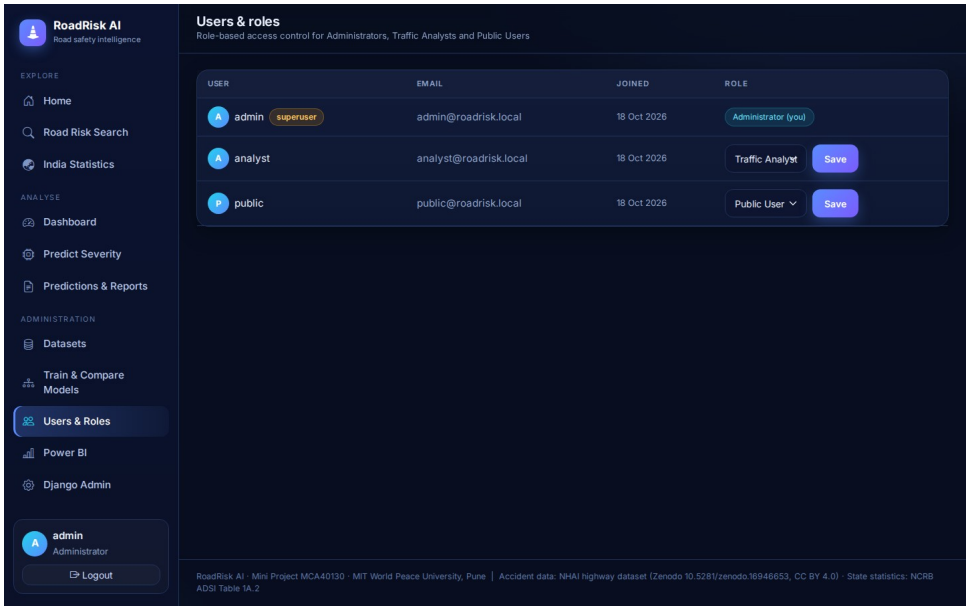


Fig 4.15 – Users and roles

4.11 India Statistics Screen (Output)

Output: real NCRB state, UT and city-wise road accident cases, injured and died, with charts and deaths per 100 accidents.



Fig 4.16 – India road accident statistics

5. TEST PROCEDURES AND IMPLEMENTATION

5.1 Implementation Approach

The system was implemented incrementally in the following phases:

1. **Data collection and preparation:** the NHAI highway accident dataset (Zenodo, CC BY 4.0) and the NCRB state statistics were collected; the accident file was decoded, de-duplicated and converted to the format of the system (details in Section 5.5).
2. **Model development:** Random Forest and XGBoost classifiers were trained on a stratified 80:20 train-test split with square-root balanced sample weights, evaluated on accuracy, precision, recall, weighted and macro F1-score, and by 5-fold cross-validation; the model with the best weighted F1-score is saved with Joblib.
3. **Backend development:** Django models, authentication, role-based permissions, and REST APIs were built for upload, prediction, analytics, and reporting.
4. **Frontend development:** a dark, responsive dashboard with a sidebar layout was built with Bootstrap and Chart.js (libraries bundled locally), covering login, prediction, dashboard, search, reports, India statistics and admin screens.
5. **Power BI integration:** an export of the same data is provided for building the executive report in Power BI Desktop.
6. **Testing and refinement:** each module was tested individually and then as an integrated system with an automated test suite.

5.2 Testing Strategy

Testing was carried out at four levels. **Unit testing** verified individual functions such as preprocessing and prediction. **Integration testing** checked the interaction between the frontend, the REST API, the ML engine, and the database. **System testing** validated the complete workflow from login to report generation. **User acceptance testing** confirmed that the screens and outputs meet the user requirements. API endpoints were additionally tested with Postman.

5.3 Test Cases

Table 5.1 – Test Cases

ID	Module	Test Input / Action	Expected Result	Status
TC01	Authentication	Register with valid details	Account created; redirected to login	Pass
TC02	Authentication	Login with wrong password	Error message shown; access denied	Pass
TC03	Access Control	Public user opens the admin panel URL	Access denied (HTTP 403)	Pass

ID	Module	Test Input / Action	Expected Result	Status
TC04	Data Management	Admin uploads a valid CSV	Data validated and stored	Pass
TC05	Data Management	Upload a CSV with missing columns	File rejected with an error message	Pass
TC06	ML Engine	Admin triggers model training	Metrics shown; best model saved	Pass
TC07	Prediction	Submit valid road conditions	Severity class returned with recommendation	Pass
TC08	Prediction	Submit form with empty fields	Validation error displayed	Pass
TC09	Road Risk Search	Search an existing road name	Historical risk profile displayed	Pass
TC10	Reporting	Click Generate Report	PDF / Excel file downloads	Pass
TC11	Power BI	Refresh the report	Visuals update from the exported data	[ ]
TC12	India Statistics	Open the India Statistics page	State, UT and city tables load; all-India total is 4,46,768 cases	Pass
TC13	Data Management	Upload a CSV that contains duplicate rows	Duplicates removed and reported	Pass

[TC11 (Power BI refresh) must be tested after the Power BI report is built.] TC01–TC10, TC12 and TC13 were verified with the automated test suite (13 tests, run with python manage.py test).

5.4 Model Evaluation

The two classifiers were evaluated on a stratified 20% hold-out test set (804 records) and by 5-fold cross-validation, using weighted accuracy, precision, recall and F1-score, plus macro F1-score because the Fatal class is rare. The model with the highest weighted F1-score is saved and used for live prediction. A majority-class baseline (always predicting Minor) is reported for comparison.

Table 5.2 – Model Performance Comparison

Model	Accuracy	Precision	Recall	F1 (weighted)	F1 (macro)	CV accuracy
Random Forest	0.555	0.515	0.555	0.524	0.371	0.570 ± 0.015
XGBoost (selected)	0.562	0.530	0.562	0.540	0.397	0.571 ± 0.018
Majority-class baseline	0.555	–	–	–	–	–

Table 5.3 – Per-Class Results of the Selected Model (XGBoost)

Class	Precision	Recall	F1-score	Test records
Minor	0.626	0.724	0.672	446
Serious	0.465	0.436	0.450	289
Fatal	0.176	0.043	0.070	69

Table 5.4 – Confusion Matrix of the Selected Model (rows = actual, columns = predicted)

Actual \ Predicted	Minor	Serious	Fatal
Minor	323	118	5
Serious	154	126	9
Fatal	39	27	3

Feature importance (XGBoost): vehicle type 26.6%, weather 17.0%, day of week 12.8%, road condition 11.3%, highway corridor 9.0%, contributing cause 8.2%, time of day 6.7%, carriageway type 5.4% and area type 3.1%.

Interpretation. Predicting severity from the recorded conditions is difficult: hold-out accuracy is only slightly above the majority-class baseline, but weighted F1-score (0.540 against about 0.40 for the baseline) and macro F1-score show that the models do learn useful patterns, mainly for the Minor and Serious classes. The rare Fatal class (about 9% of records) is detected poorly (recall 4%), so the system should be used as a decision-support estimate rather than a precise forecast. The authors of the source dataset report a similarly modest accuracy (about 41% on four classes) in their own study. The values above are from the training run shown in Fig 4.4; re-training on another machine may change the last decimal place.

5.5 Dataset and Preprocessing

The accident records come from the dataset “Accident Data of Selected Indian Highways for Accident Severity Prediction Using Machine Learning Models” (Khanum, Kulkarni, Garg and Faheem, Zenodo, 2025, doi:10.5281/zenodo.16946653, licence CC BY 4.0), which contains accident records provided by NHAI concessionaires for four highway projects. The raw file was converted with the script tools/prepare_nhai_dataset.py:

- Numeric codes (severity, weather, vehicle type, road condition, carriageway type, area type) were decoded with the code tables published by the dataset authors in their F1000Research paper.
- The raw file lists every record twice; exact duplicates were removed (8,116 rows reduced to 4,053 unique rows), and two stray records dated 2003 were dropped. Duplicates are also removed on every upload, because identical records in both the training and test sets would inflate the results.

- Records were assigned to highway corridors from the chainage and year, and 10-km road stretches (for example Pune-Solapur NH-9 km 180–190) were built from the chainage. The Nagpur and Chengapally–Walayar records (2022–23) cannot be separated in the source file and form one group.
- The weekday was derived from the date, because the raw day-of-week code is inconsistent between the older and the 2022–23 records. Time was grouped into Morning, Afternoon, Evening and Night.
- The four injury classes were grouped into three: Fatal, Serious (grievous injury) and Minor (minor injury or no injury).
- Causes coded 4 to 8 in the source file could not be mapped individually and were grouped as “Other (driver / vehicle / road fault)”.

Table 5.5 – Records Used by the System (after preprocessing: 4,020 records, 2013–2023)

Corridor	Records	Fatal	Serious	Minor
Pune–Solapur (NH-9), Maharashtra	1,383	116	430	837
Barwa–Adda–Panagarh (NH-2), Jharkhand and West Bengal	1,840	153	777	910
Nagpur and Chengapally–Walayar (2022–23)	797	77	237	483
All corridors	4,020	346	1,444	2,230

6. SAMPLE CODE

The following extracts show the core parts of the implementation: the Django models, the model training and selection routine, and the prediction view and API (core/models.py, core/ml.py, core/views.py).

6.1 Django Models (core/models.py)

```
from django.contrib.auth.models import AbstractUser
from django.db import models


class User(AbstractUser):
    ROLE_CHOICES = [("admin", "Administrator"), ("analyst", "Traffic Analyst"),
                    ("public", "Public User")]
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default="public")


class Accident(models.Model):
    dataset = models.ForeignKey(Dataset, null=True, on_delete=models.CASCADE)
    date = models.DateField(null=True, blank=True)
    road_name = models.CharField(max_length=150, db_index=True)
    corridor = models.CharField(max_length=80, default="Unknown", db_index=True)
    area_type = models.CharField(max_length=30, default="Unknown")
    road_type = models.CharField(max_length=40, default="Unknown")
    road_condition = models.CharField(max_length=40, default="Unknown")
    weather = models.CharField(max_length=30)
    time_of_day = models.CharField(max_length=20)
    day_of_week = models.CharField(max_length=12, default="Unknown")
    vehicle_type = models.CharField(max_length=40)
    cause = models.CharField(max_length=60, default="Unknown")
    severity = models.CharField(max_length=10)


class Prediction(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    road_name = models.CharField(max_length=150, blank=True)
    conditions = models.JSONField(default=dict)
    predicted_severity = models.CharField(max_length=10)
    confidence = models.FloatField(default=0)
    probabilities = models.JSONField(default=dict, blank=True)
    model_used = models.CharField(max_length=40)
    created_at = models.DateTimeField(auto_now_add=True)
```

6.2 Model Training and Selection (core/ml.py)

```
def _fit(model, X, y):
    # square-root balanced sample weights: the rare Fatal class counts more
    weights = compute_sample_weight("balanced", y) ** 0.5
    model.fit(X, y, sample_weight=weights)
    return model


def train_models(user=None):
    df = pd.DataFrame.from_records(approved_accidents().values(*FEATURES, "severity"))
    features = usable_features(df)          # features with at least 2 known values
    X = pd.get_dummies(df[features].astype(str))
```

```
    encoder = LabelEncoder().fit(df["severity"])
    y = encoder.transform(df["severity"])
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=42)

    candidates = {
        "Random Forest": RandomForestClassifier(
            n_estimators=300, min_samples_leaf=3, random_state=42, n_jobs=-1),
        "XGBoost": XGBClassifier(
            n_estimators=250, max_depth=4, learning_rate=0.05, subsample=0.9,
            colsample_bytree=0.9, eval_metric="mlogloss", random_state=42),
    }
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    results = {}
    for name, model in candidates.items():
        _fit(model, X_train, y_train)
        pred = model.predict(X_test)
        results[name] = {
            "model": model,
            "accuracy": accuracy_score(y_test, pred),
            "precision": precision_score(y_test, pred, average="weighted",
zero_division=0),
            "recall": recall_score(y_test, pred, average="weighted", zero_division=0),
            "f1": f1_score(y_test, pred, average="weighted", zero_division=0),
            "cv_scores": _cv_accuracy(model, X, y, cv),
        }

    best = max(results, key=lambda k: results[k]["f1"])
    joblib.dump({"model": results[best]["model"], "columns": list(X.columns),
                 "encoder": encoder, "name": best, "features": features}, MODEL_PATH)
    return best, results
```

6.3 Prediction View and Recommendations (core/views.py)

```
@login_required
def predict_view(request):
    if not ml.model_ready():
        return render(request, "core/no_model.html")
    form = PredictForm(request.POST or None)      # fields built from the trained model
    if request.method == "POST" and form.is_valid():
        cond = {f: form.cleaned_data[f] for f in form.model_features}
        label, probs, model_name = ml.predict(cond)
        pred = Prediction.objects.create(
            user=request.user, road_name=form.cleaned_data["road_name"],
            conditions=cond, predicted_severity=label,
            confidence=probs[label], probabilities=probs, model_used=model_name)
        for text in build_recommendations(label, cond):
            Recommendation.objects.create(prediction=pred, suggestion_text=text)
        return redirect("prediction_detail", pk=pred.pk)
    return render(request, "core/predict.html", {"form": form})


def predict(conditions):                          # core/ml.py
    bundle = load_bundle()
    row = pd.DataFrame([{f: str(conditions[f]) for f in bundle["features"]}])
    X = pd.get_dummies(row).reindex(columns=bundle["columns"], fill_value=0)
    proba = bundle["model"].predict_proba(X)[0]
    probs = dict(zip(bundle["encoder"].classes_, map(float, proba)))
    return max(probs, key=probs.get), probs, bundle["name"]
```

7. LIMITATIONS

- Severity prediction from the recorded conditions is inherently difficult: hold-out accuracy (about 56%) is only marginally above the majority-class baseline (55.5%), and the rare Fatal class (about 9% of records) is detected poorly (recall about 4%). The predictions are probabilistic estimates, not certainties.
- The model is trained on a published dataset of four NHAI highway projects (4,020 records after preprocessing), not on all roads of India. Predictions for other regions, urban roads or road types not present in the data are less reliable. India-wide figures (NCRB) are summary totals and are used only for statistics, not for training.
- Accident-level data with lighting, junction type, traffic volume and speed was not available in the source file, so these factors are not used. The contributing cause is known only after an accident, so it is used here as a what-if input.
- The Nagpur and Chengapally–Walayar records cannot be separated in the source file; the causes coded 4 to 8 could not be mapped individually; and the four injury classes were reduced to three (No injury was grouped with Minor).
- The system works on historical data and does not use real-time inputs such as live traffic, GPS, or sensor feeds.
- AI recommendations are rule-based on the predicted severity and the input conditions, and should be reviewed by domain experts before being applied.
- Power BI reports require Power BI Desktop to build, and a licensed service for sharing and scheduled refresh.

8. FUTURE SCOPE

- Integration of real-time GPS, IoT sensors, and live traffic-camera analysis for instant risk alerts.
- Use of deep learning and geospatial models for finer location-level prediction.
- Interactive map-based visualisation of accident black spots using GIS tools.
- A mobile application and push notifications that warn drivers about high-risk stretches.
- Integration of live weather and traffic APIs to automatically populate prediction inputs.
- Multi-language support and deployment on a cloud platform for use by state transport departments.
- Explainable AI techniques such as SHAP to show which factors drive each severity prediction.

9. CONCLUSION

RoadRisk AI demonstrates how machine learning, web development, and business intelligence can be combined into a single deployable platform for road safety. Using real NHAI highway accident records, with the Pune–Solapur (NH-9) corridor as the focus, the system lets administrators upload and manage datasets, trains and compares Random Forest and XGBoost models, and uses the best model to predict accident severity as Fatal, Serious, or Minor. An interactive dashboard, a road risk search for 10-km stretches, an India-wide statistics page based on NCRB data, downloadable reports, and a Power BI executive layer turn raw accident data into actionable insight, while the AI recommendation module supports preventive decision-making.

The evaluation was kept honest: severity prediction on the available real data reaches an accuracy of about 56% and a weighted F1-score of 0.54, only modestly above a majority-class baseline, and the rare Fatal class is hard to detect. This reflects the limits of the recorded factors rather than of the software, and is discussed in Section 7. The layered architecture, role-based access control, relational database, and REST APIs make the system extensible, and it can be improved with richer data, real-time feeds and advanced models in the future. The project helped me apply software engineering, database design, machine learning and data visualisation to a socially relevant problem.

10. REFERENCES

1. H. Khanum, R. Kulkarni, A. Garg and M. I. Faheem, “Accident Data of Selected Indian Highways for Accident Severity Prediction Using Machine Learning Models,” Zenodo, 2025, doi:10.5281/zenodo.16946653 (CC BY 4.0).
2. Accident severity prediction modeling for road safety using the random forest algorithm: an analysis of Indian highways, F1000Research, 2023, 12:494 (code tables for the dataset). [Add the full author list from the article page.]
3. National Crime Records Bureau (NCRB), Ministry of Home Affairs, Government of India, “Accidental Deaths & Suicides in India (ADSI),” Table 1A.2: Traffic Accidents (State/UT/City-wise). [Add the report year and URL of the file you downloaded.]
4. Ministry of Road Transport and Highways, Government of India, “Road Accidents in India,” Annual Report.
5. L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001.
6. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
7. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, 2011.
8. Django Software Foundation, Django Documentation, https://docs.djangoproject.com
9. Encode, Django REST Framework Documentation, https://www.django-rest-framework.org
10. Microsoft, Power BI Documentation, https://learn.microsoft.com/power-bi/